

Quick Sort

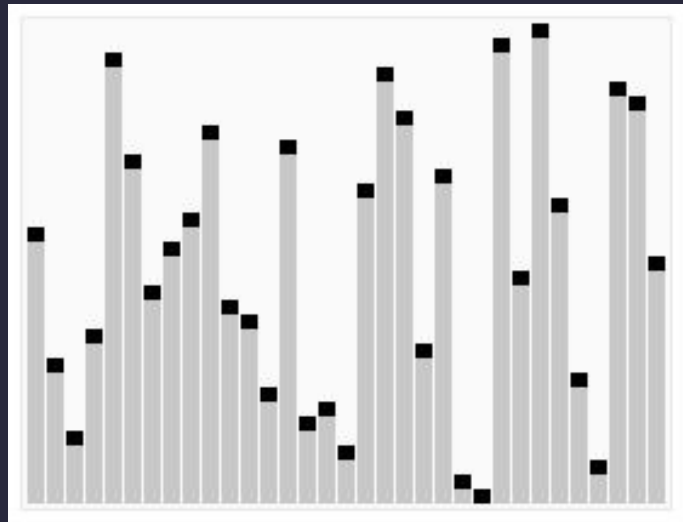
ALGORITMO DE ORDENAÇÃO

Resolução de Problemas
Estruturados em Computação



/CONCEITO GERAL

O **quicksort** adota a estratégia divisão e conquista. A estratégia consiste em rearranjar as chaves de modo que as chaves "menores" precedam as chaves "maiores". Em seguida o quicksort ordena as duas sublistas de chaves menores e maiores recursivamente até que a lista completa se encontre ordenada.

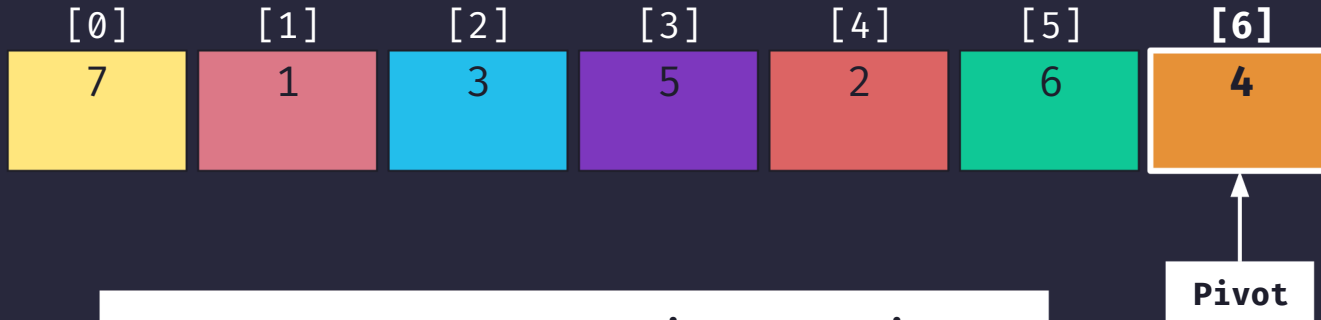


/ORDENANDO ELEMENTOS

[0]	[1]	[2]	[3]	[4]	[5]	[6]
7	1	3	5	2	6	4

O primeiro passo é definir um elemento que chamaremos de pivot. Como exemplo, vamos pegar o último elemento do array

/ORDENANDO ELEMENTOS

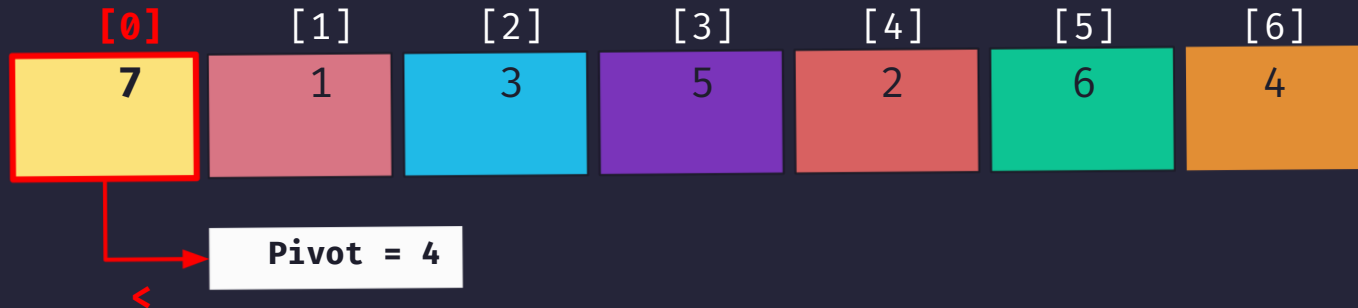


Agora que temos nosso pivot, precisamos encontrar sua posição correta (índice) e colocar os menores valores à esquerda e os maiores à direita

Basicamente vamos dividir o nosso array, a partir da posição correta do pivot, em dois subarrays, um com os valores menores que o pivot e outro com os valores maiores.

Chamaremos cada subarray de partição

/ORDENANDO ELEMENTOS



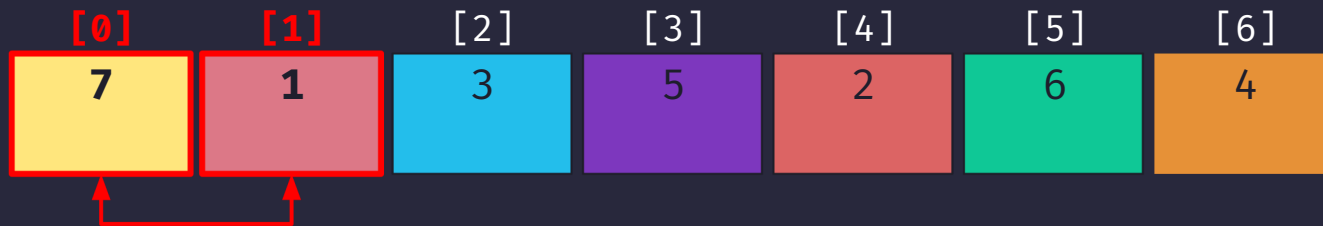
Vamos começar avaliando pelo começo do array. 7 é menor que o pivot?
Não! Então vamos para a próxima posição.

/ORDENANDO ELEMENTOS



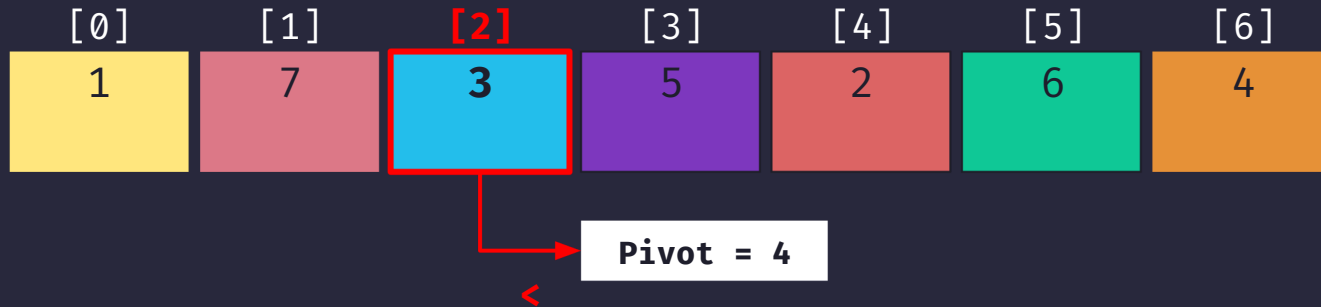
1 é menor do que o pivot? Sim! Então vamos colocá-lo mais à esquerda. Como?

/ORDENANDO ELEMENTOS



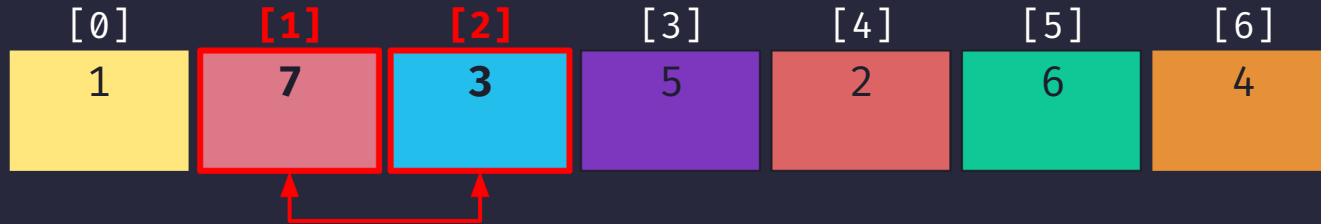
Trocando o 1 (posição [1]) pelo valor 7 (posição [0]). Assim colocamos ele na primeira posição disponível da partição dos menores (esquerda)

/ORDENANDO ELEMENTOS



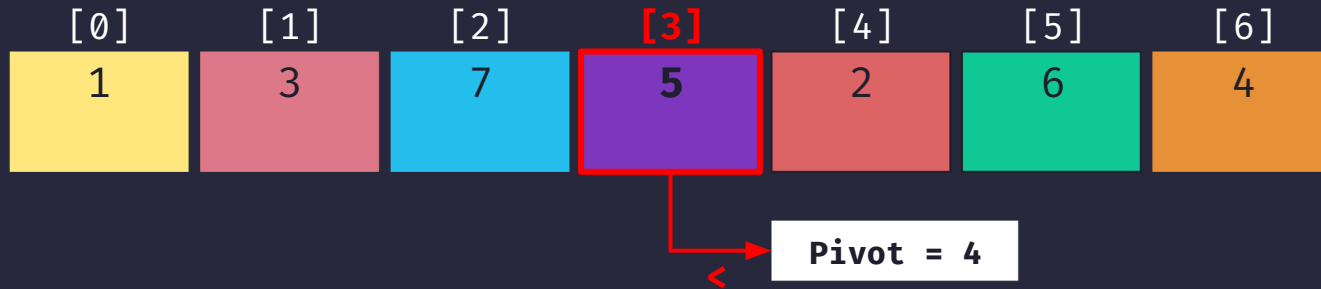
Depois de trocar nossos valores da posição [0] e [1] seguimos para a próxima posição. 3 é menor que o pivot? Sim! Então vai para a esquerda. Como?

/ORDENANDO ELEMENTOS



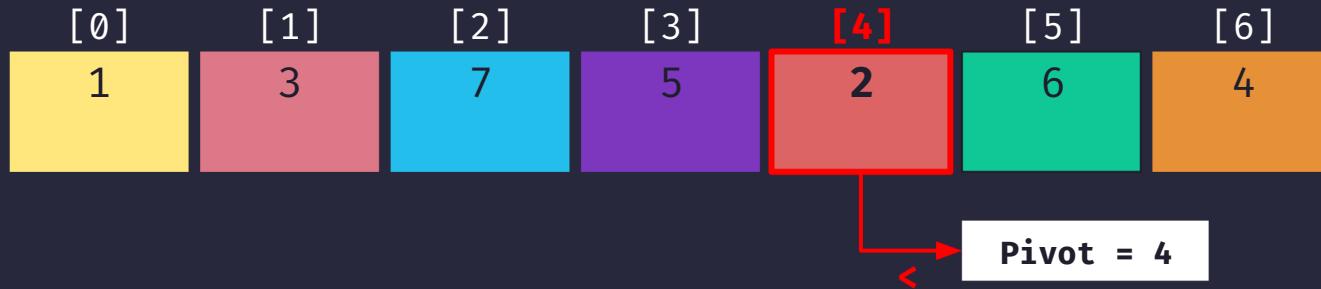
Trocando o 3 (posição [2]) pelo valor 7 (posição [1]). Assim colocamos ele na primeira posição disponível da partição dos menores (esquerda). Lembrando que a posição [0] já está ocupada pelo 1 que é menor, então consideramos que não está disponível

/ORDENANDO ELEMENTOS



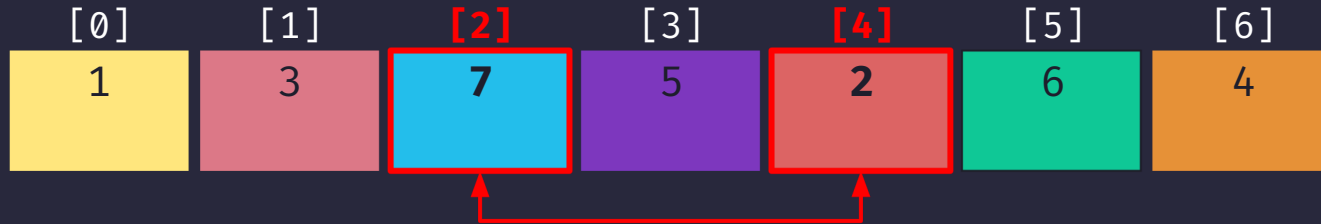
Depois de trocar nossos valores da posição [1] e [2] seguimos para a próxima posição. 5 é menor que o pivot? Não! Então vamos para próxima posição

/ORDENANDO ELEMENTOS



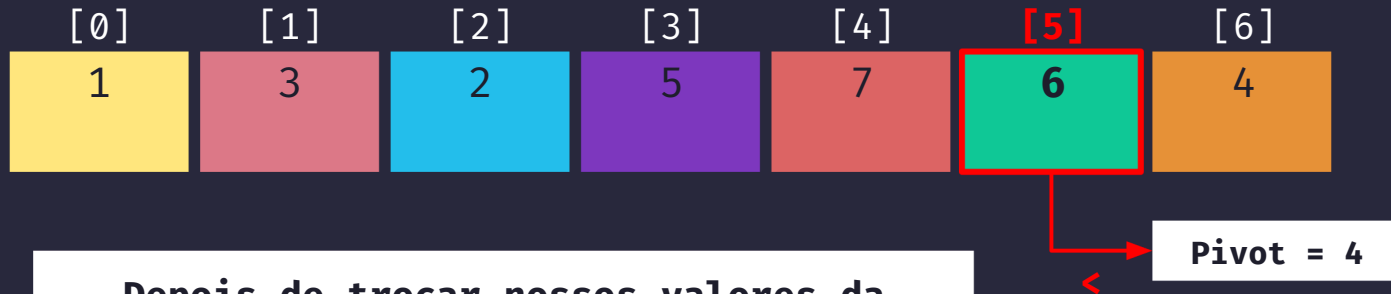
2 é menor do que o pivot? Sim! Então vamos colocá-lo mais à esquerda. Como?

/ORDENANDO ELEMENTOS



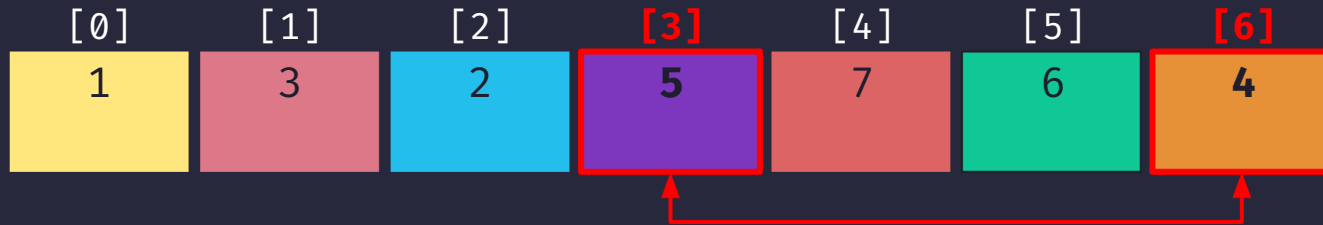
Trocando o 2 (posição [4]) pelo valor 7 (posição [2]). Assim colocamos ele na primeira posição disponível da partição dos menores (esquerda). Lembrando que o 1 e o 3, que são menores, já estão ocupando as posições [0] e [1]

/ORDENANDO ELEMENTOS



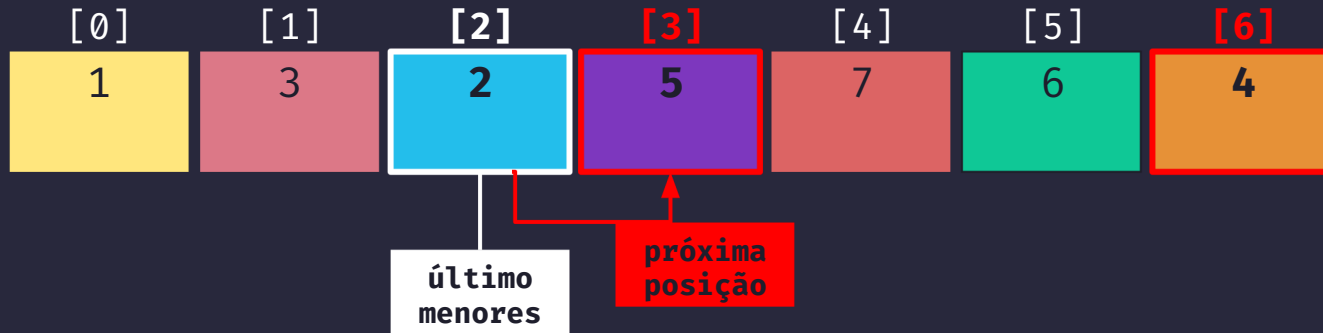
Depois de trocar nossos valores da posição [4] e [2] seguimos para a próxima posição. 6 é menor que o pivot?
Não! Mas não vamos para a próxima posição pois é a posição do pivot

/ORDENANDO ELEMENTOS



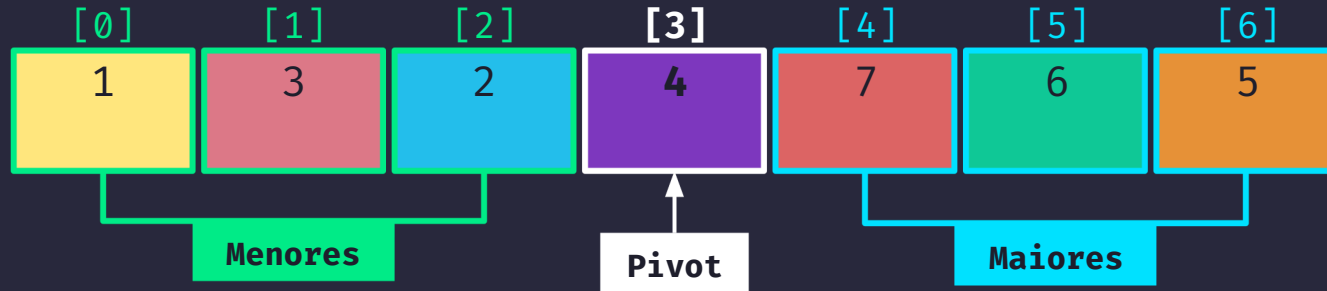
O que vamos fazer é colocar o nosso pivot no local correto. Trocando o 4 (posição [6]) pelo valor 5 (posição [3]).

/ORDENANDO ELEMENTOS



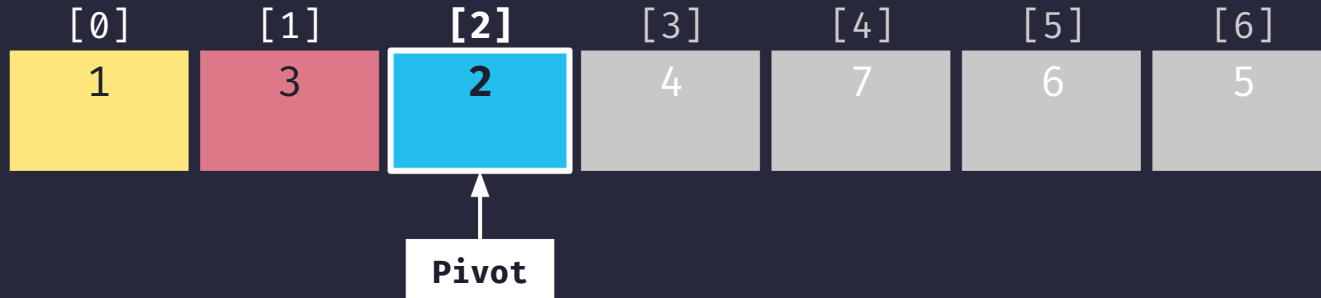
Nós conseguimos encontrar essa posição pois ela será sempre a próxima posição disponível da partição dos menores (esquerda)

/ORDENANDO ELEMENTOS



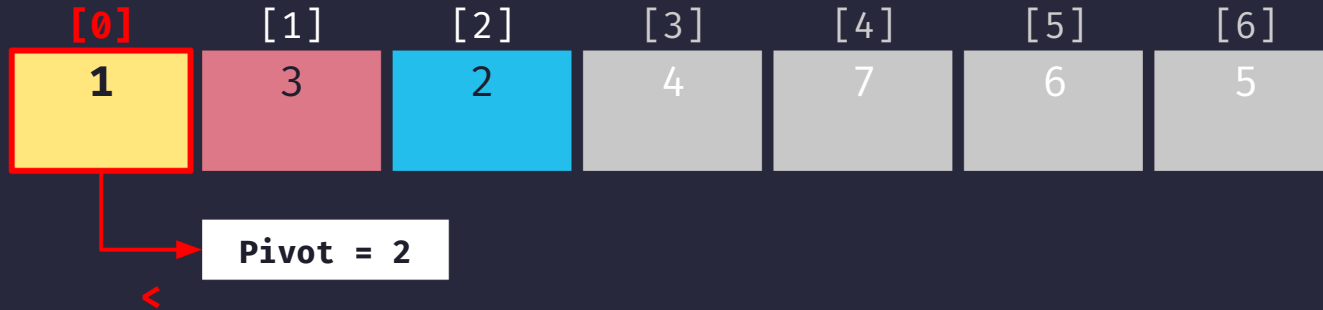
Com isso conseguimos colocar o nosso pivot no local correto do array e fizemos duas partições. À esquerda os menores que o pivot e à direita os maiores que o pivot

/ORDENANDO MENORES (PT 1)



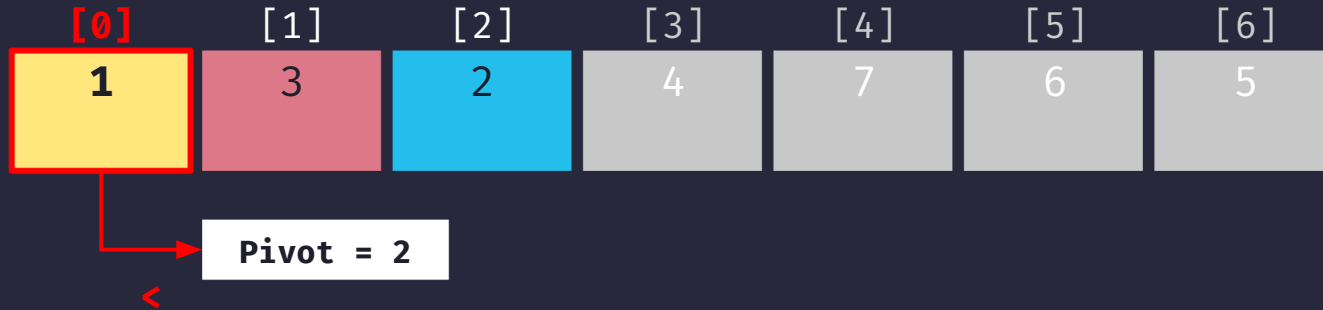
Agora vamos repetir o processo para ordenar a partição dos menores. Começamos definindo o pivot como sendo o último elemento.

/ORDENANDO MENORES (PT 1)



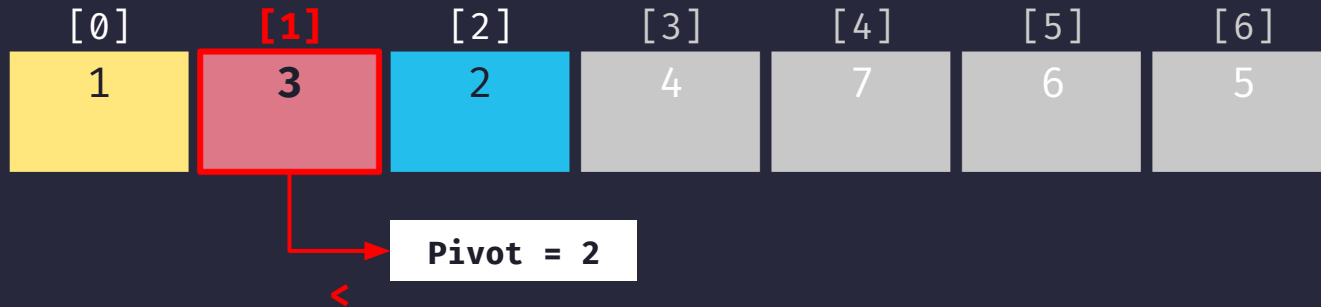
Agora vamos avaliar pelo começo do array.
1 é menor que o pivot?
Sim! Então vamos colocá-lo mais à esquerda

/ORDENANDO MENORES (PT 1)



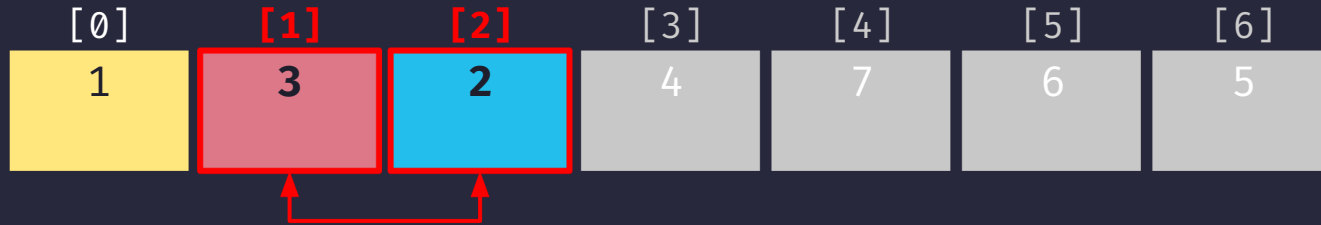
Nesse caso o algoritmo vai fazer a troca do 1 na posição [0] com o próprio 1 da posição [0]. Mantendo ele na primeira posição disponível da partição dos menores

/ORDENANDO MENORES (PT 1)



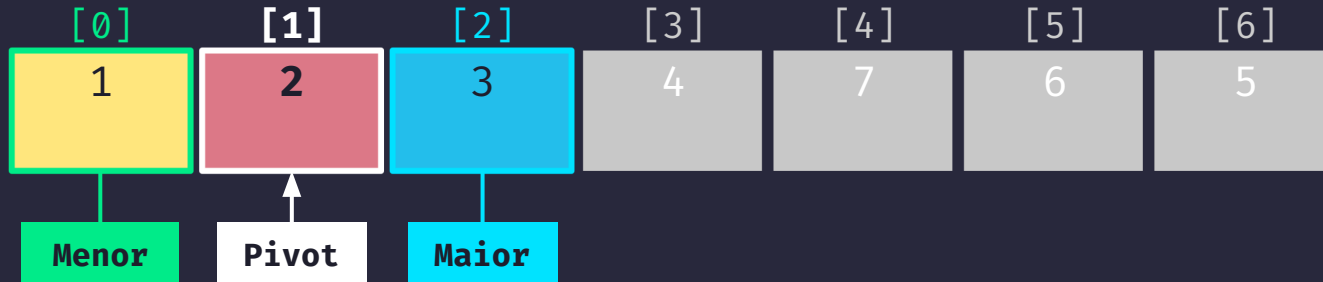
Depois de trocar nosso valor por ele mesmo (😞) seguimos para a próxima posição. 3 é menor que o pivot? Não! Mas não vamos para a próxima posição pois é a posição do pivot

/ORDENANDO MENORES (PT 1)



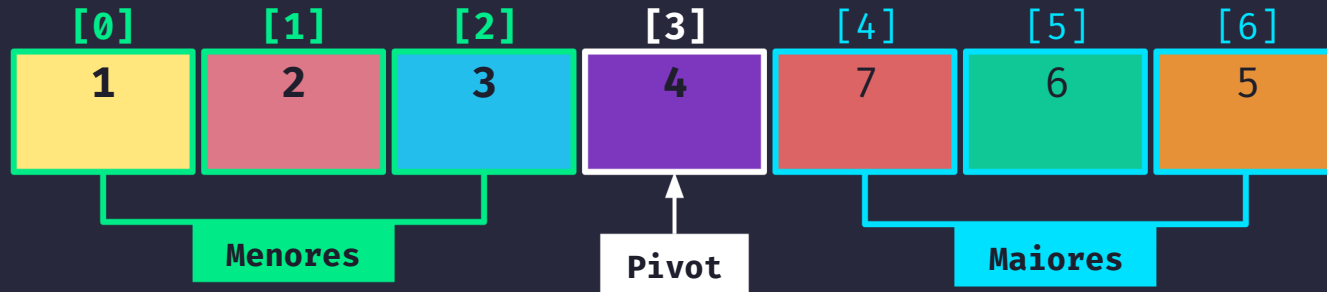
O que vamos fazer é colocar o nosso pivot no local correto. Trocando o 2 (posição [2]) pelo valor 3 (posição [1]). Lembrando que sabemos a posição correta do pivot pois será sempre a próxima posição disponível da partição dos menores (esquerda)

/ORDENANDO MENORES (PT 1)



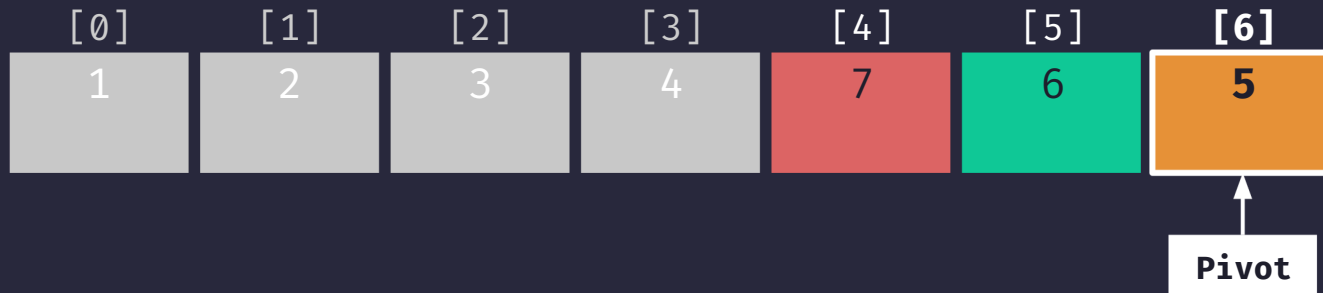
Com isso conseguimos colocar o nosso pivot no local correto do array mas não há necessidade de criar partições pois só temos 1 elemento menor e 1 maior. Caso houvessem mais elementos iríamos repetir o processo para as partições da partição(😓)

/ORDENANDO ELEMENTOS



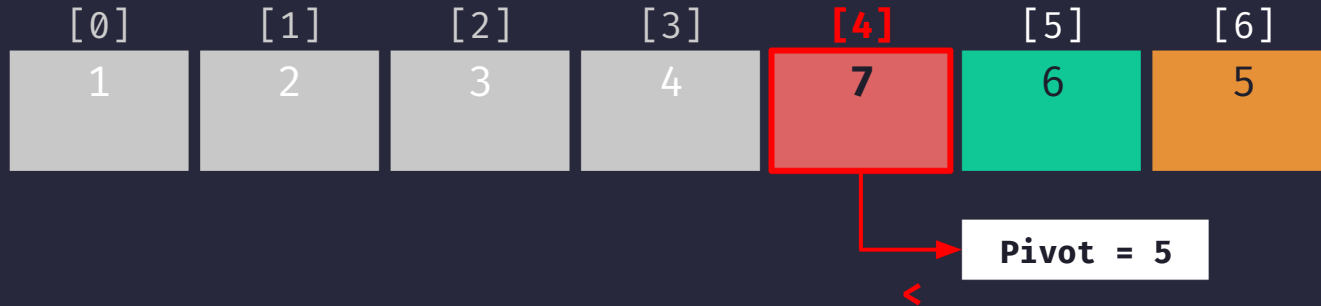
Agora retornamos ao nosso array principal com a nossa partição de menores (esquerda) já ordenado. Vamos repetir o processo com a partição dos maiores (direita).

/ORDENANDO MAIORES (PT 2)



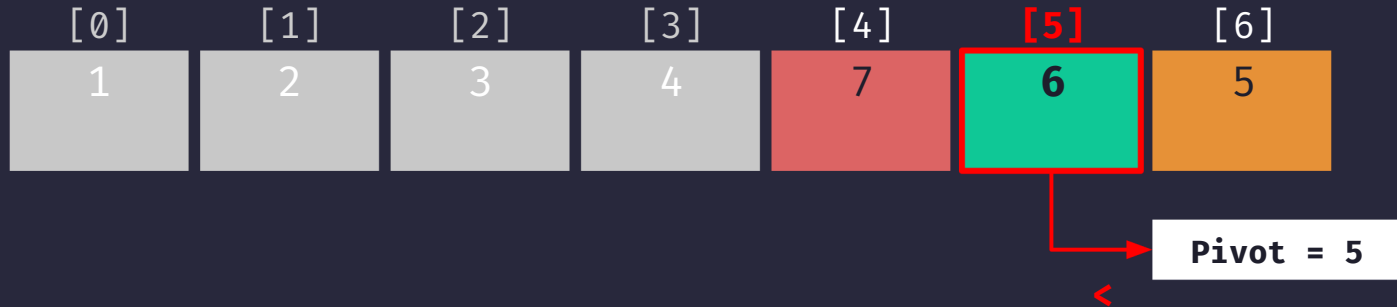
Começamos como sempre, definindo nosso pivot com o valor do último elemento do array.

/ORDENANDO MAIORES (PT 2)



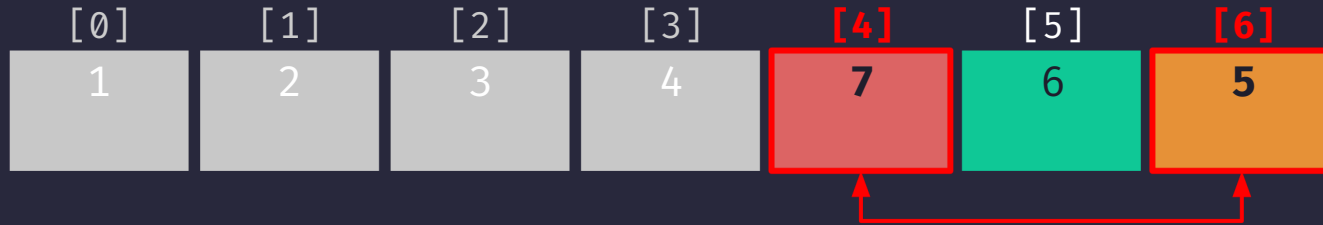
Agora vamos avaliar pelo começo do array.
7 é menor que o pivot?
Não! Então vamos para a próxima posição

/ORDENANDO MAIORES (PT 2)



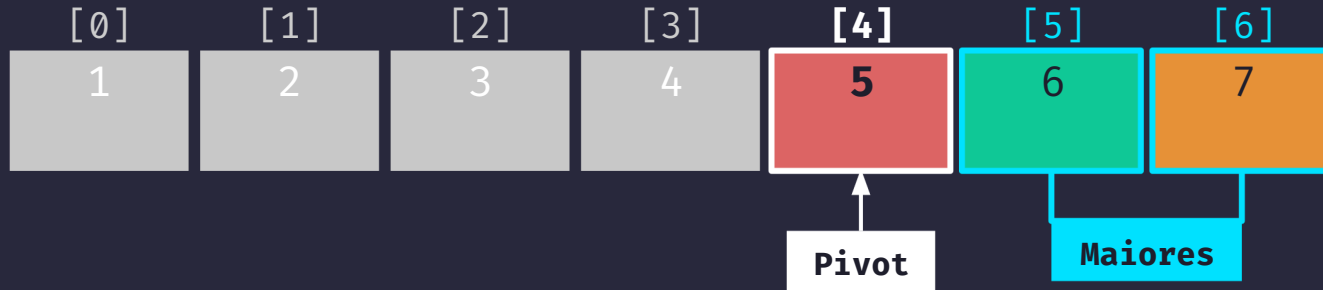
6 é menor do que o pivot? Não! Mas não vamos para a próxima posição pois é a posição do pivot

/ORDENANDO MAIORES (PT 2)



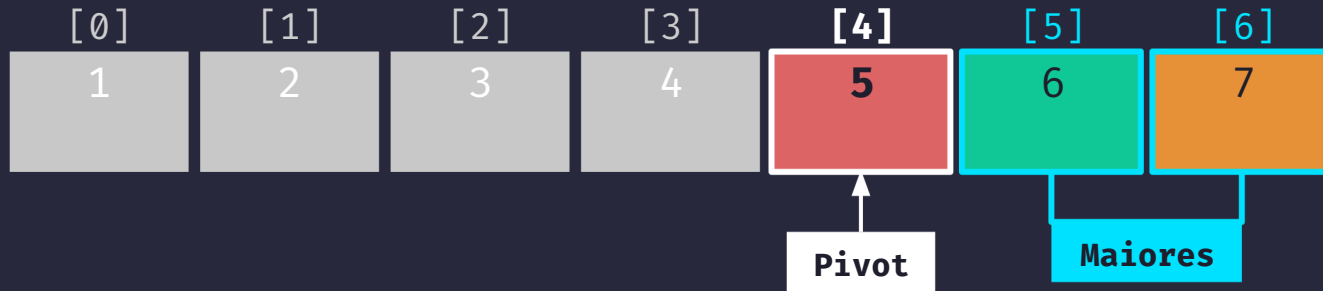
O que vamos fazer é colocar o nosso pivot no local correto. Trocando o 5 (posição [6]) pelo valor 7 (posição [4]). Lembrando que sabemos a posição correta do pivot pois será sempre a próxima posição disponível da partição dos menores (esquerda)

/ORDENANDO MAIORES (PT 2)



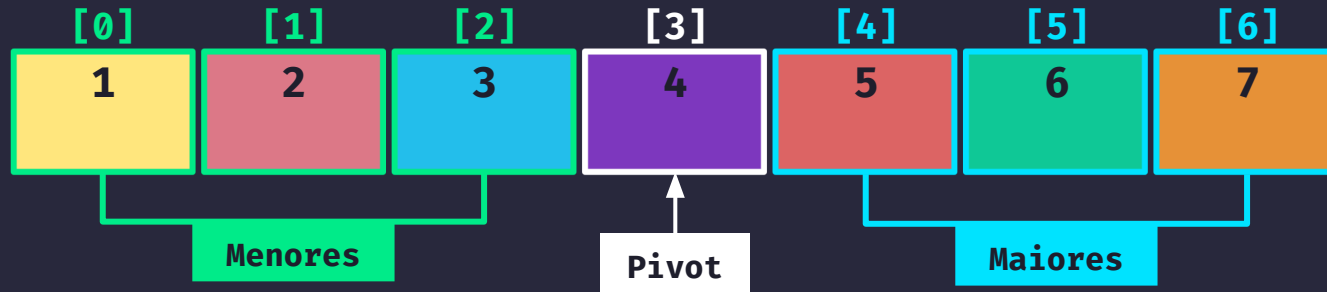
Com isso conseguimos colocar o nosso pivot no local correto do array. Nesse caso, acabamos tendo uma partição de maiores, pois ela possui dois elementos. Visualmente sabemos que os elementos estão corretos, mas o algoritmo irá realizar o processo novamente

/ORDENANDO MAIORES (PT 2)



A essa altura do campeonato eu imagino que vocês já tenham entendido que o mesmo processo foi realizado com a partição dos maiores, tendo 7 como pivot, verificando se o 6 era menor que o pivot, trocando o 6 de posição com ele mesmo e retornando o array ordenado para a chamada anterior.

/ORDENANDO ELEMENTOS



Agora retornamos ao nosso array principal com a nossa partição de maiores (direita) já ordenada. E fim! 🥰